RAW 归档（RAW Archiver）源代码与开发说明

版本：v1.0.0 内部标识：rawarchiver 状态：开发中（未上线）

第一部分 产品开发说明

1. 产品定位

RAW 归档是一款面向 NAS（群晖 DSM / 绿联 fnOS）的相机照片日期整理工具。核心能力：从 U 盘 / SD 卡 / TF 卡直接读取照片与视频，解析拍摄日期（EXIF），按 YYYYMMDD 目录结构归档到用户指定的任意 NAS 目录，并生成机器可消费的清单报告。全程以「源只读、目标可写、可断点续跑、可清源」为设计准则。

2. 核心流程（定稿）

1. 选源：Web 界面列出挂载在 /volumeUSB*/usbshare 的可移动设备（卷标、容量、可识别文件数），也支持手填任意本地目录。
2. 选输出目录：每次运行时自由指定归档落点（如 /volume1/photos/2026-10-黄山），目录不存在自动创建；非法路径、无写权限、输出落在源内部均会被拦截。
3. 预览确认：只扫描不动盘，展示文件数/容量/日期跨度、日期来源、仅能靠文件时间推断日期的告警项、同名冲突数。
4. 执行：默认启用「内部暂存」——先把卡顺序整卷复制到 NAS 临时目录（shutil.copy2 保留 mtime，保证视频兜底日期准确），再就地按 YYYYMMDD/原名_YYYYMMDD.ext 归档（同卷 rename，秒级）。
5. 清源（move 模式）：全部归档成功后，按 state.json 的待删清单逐条校验大小并删除卡上源文件，回收空目录；@eaDir 保留不动。
6. 落报告：在输出目录根写 _report_<taskId>.json，下游只消费该报告与输出目录，卡可随时拔走。

3. 源类型与模式约定

源类型	模式	行为
可移动（卡 / U 盘）	copy	复制，卡不动
可移动	move	复制 -> 校验 -> 删卡上源文件 + 收空目录
本地同卷	copy	复制
本地同卷	move	shutil.move 原子 rename，可回滚到 _restored/

跨卷没有真正的 move（底层仍是复制+删除，不原子），因此卡源 move 严格拆为「顺序暂存 -> 同卷归档 -> 全部成功后按清单清源」三段，任一环节断电，最坏情况是源和目标各有一份，只多不少、不丢片。

4. 关键设计

- 日期解析四级兜底：exifread（RAW/JPG 主力）-> Pillow（JPG 备用）-> ExifTool（视频主力、RAW 兜底）-> 文件 mtime（仅视频允许，照片无 EXIF 视为 unknown 并跳过）。全程纯读，拿不到日期返回 (None, "unknown")，绝不抛异常。
- 内部暂存加速：卡上逐张读 EXIF 是大量随机小读，慢且发热。暂存模式下先顺序整卷复制到 NAS 再就地处理，对使用者完全透明，速度快数倍。
- 断点自包含：任务态写在 <输出目录>/.rawarchiver/tasks/<taskId>/state.json，换机器 / 重建 data / 半年后重插同一张卡补漏，幂等判断依然准确。
- 命名与冲突：目标名 原名_YYYYMMDD.ext，同日同目录合并；同名冲突自动追加 _1/_2；可选品牌分层、月份平铺目录（月份副本仅 JPG+视频）。
- 只读安全线：源永远只读，除非 move 模式且全部归档成功才清源；输出目录不能落在源内部（防自拷贝死循环）。

5. 命名规范

层级	名称
中文名（标题/文档/界面）	RAW 归档
英文名（包名/进程/路径/URL）	rawarchiver
主脚本	raw_archiver.py
版本号	1.0.0
状态目录	.rawarchiver/（输出目录下）、data/rawarchiver/
报告文件	_report_<taskId>.json
产物	RAWArchiver-1.0.0.spk / .fpk
URL 前缀	/app/rawarchiver/

规则：中文只出现在 title、文案、文档；路径 / 包名 / 脚本 / URL 一律小写 ASCII，无空格无中文，避免 DSM 的 Apache/Perl 及 fnOS 打包工具在编码上出问题。

6. 项目目录结构

```
rawarchiver/
├── README.md / requirements.txt / build_all.sh
├── app/rawarchiver/
│   ├── www/index.html # Web 面板（源/输出/预览/进度）
│   └── api/index.cgi # DSM CGI 入口
```

├── scripts/

│ ├── raw_archiver.py # 核心

│ ├── setup.py / pipeline.sh / hooks/post_done.sh

│ └── fetch_exiftool.sh # 拉取/复制 ExifTool

├── agent/agent.py # 本机 FastAPI 版（8765 端口）

├── pkg/spk/ # DSM 套件：INFO / installer / uninstaller

├── pkg/fpk/ # fnOS 包：package.conf + lifecycle

└── vendor/exiftool/ # ExifTool（离线出包）

7. 部署与权限要点

- CGI 读 USB 权限：DSM 下 CGI 以 http 用户运行，常能读 /volumeUSB1/usbshare，但若挂载为 root:root 700 会扫不到。可先在 SSH 用 sudo -u http ls /volumeUSB1/usbshare 探测。
- 清卡必须 root：unlink 卡上文件常因挂载属主报 Permission denied，表现为「归档成功但卡没清掉」。稳妥做法是走 agent.py（本机/root 身份）或挂到「任务计划程序」以 root 跑 pipeline。
- @eaDir 必须跳过：Synology 会在每个目录生成 @eaDir 缩略图，已在跳过名单，否则会被当图片扫入。
- 换口续跑：断点后重插尽量插同一个挂载点；换口后清理会拒绝执行，靠 _report_*.json 里的 src+size 人工核对，或 --cleanup 指定正确路径补清。

8. 已修复的关键缺陷（合并版实测复验）

1. 目标路径不再带上源的 DCIM/100CANON 层级，扁平为 YYYYMMDD/原名_YYYYMMDD.ext。
2. --cleanup / --rollback 曾被「缺少 source」提前拦掉，已调整 CLI 分支顺序，清源现在可正常执行。
3. 可移动介质已清源才禁回滚；本地目录 move 允许还原到 _restored/。
4. 重跑同一任务时 skipped/failed 不再跨轮累加。
5. --info 在 overlay 挂载上把容量算成 2^64 荒谬值，已加溢出保护。
6. shell 脚本续行反斜杠曾被写成双反斜杠（build_all.sh 直接语法报错），已全量修正并 bash -n 全绿。

第二部分 源代码

scripts/raw_archiver.py（核心处理脚本）

RAW 归档（RAW Archiver）
第二部分 · 源代码（更正版）

本部分替换原《RAW归档_源码与开发说明.pdf》第 4–13 页的旧源码清单。
以下源码为 RAWArchiver/ 当前实际代码，已按《第二部分源代码-更正清单》落地：

· #1 buil

scripts/raw_archiver.py（核心处理脚本）

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
RAW 归档 —— 核心处理脚本 v1.0.0
源：U盘/SD/TF卡（只读，move 模式归档后清源）或本地目录
输出：用户指定的任意 NAS 目录（自包含断点态 + 报告）
"""
import os
import sys
import json
import time
import shutil
import argparse
import subprocess
import hashlib
from datetime import datetime
from pathlib import Path
from collections import defaultdict
import re

try:
    import exifread
except Exception:
    exifread = None
try:
    from PIL import Image
except Exception:
    Image = None

APP_NAME = 'RAW 归档'
APP_DIR  = '.rawarchiver'          # 输出目录下的自包含状态目录
APP_VER  = '1.0.0'

RAW_EXTS = {'.nef','.nrw','.cr2','.cr3','.crw','.arw','.srf','.sr2','.raf','.orf','.dng','.pef','.rw2'}
JPG_EXTS = {'.jpg','.jpeg'}
VID_EXTS = {'.mp4','.mov','.mts','.m2ts','.avi','.mkv','.3gp','.mpg','.mpeg','.wmv','.flv','.m4v','.ts','.mod','.mxf'}
ALL_EXTS = RAW_EXTS | JPG_EXTS | VID_EXTS

DATE_DIR_RE   = re.compile(r'^\d{8}$')
SKIP_DIRS     = {'@eaDir', APP_DIR, '_inbox', '_exports', '_trash', '_restored',
                 'Thumbs.db', '.DS_Store', 'lost+found'}
REMOVABLE_PFX = ('/volumeUSB', '/mnt/usb', '/media/', '/run/media/', '/Volumes/', '/mediausb')

BRAND_MAP = {
    'NIKON':'Nikon','CANON':'Canon','SONY':'Sony','OLYMPUS':'Olympus','OM DIGITAL SOLUTIONS':'Olympus',
    'PANASONIC':'Panasonic','FUJIFILM':'Fujifilm','PENTAX':'Pentax','RICOH':'Pentax','SIGMA':'Sigma',
    'LEICA':'Leica','HASSELBLAD':'Hasselblad','PHASE ONE':'PhaseOne','SAMSUNG':'Samsung',
    'GOPRO':'GoPro','DJI':'DJI','APPLE':'Apple','HUAWEI':'Huawei','XIAOMI':'Xiaomi',
}

# ======================== 工具 ========================

def now():
    return time.time()

def atomic_write(path, obj):
    p = Path(path)
```

```
    p.parent.mkdir(parents=True, exist_ok=True)
    tmp = p.with_suffix('.tmp.%d' % (time.time() * 1e6 % 1e6))
    with open(tmp, 'w', encoding='utf-8') as f:
        json.dump(obj, f, ensure_ascii=False, indent=2)
        f.flush()
        os.fsync(f.fileno())
    os.replace(tmp, p)

def log_msg(st, msg):
    st.setdefault('logs', []).append({'t': datetime.now().strftime('%H:%M:%S'), 'msg': msg})
    if len(st['logs']) > 2000:
        st['logs'] = st['logs'][-1500:]
    print(msg, flush=True)

# ---- 源类型判定 ----

def is_removable(path):
    if not path:
        return False
    p = os.path.realpath(os.path.abspath(path))
    if any(p.startswith(x) for x in REMOVABLE_PFX):
        return True
    try:
        with open('/proc/mounts', encoding='utf-8', errors='ignore') as f:
            for ln in f:
                parts = ln.split()
                if len(parts) >= 2 and p.startswith(parts[1]):
                    return any(k in ln for k in ('usb', 'udisks', 'shortname=', 'vfat', 'exfat', 'fuseblk'))
    except Exception:
        pass
    return False

def source_info(path):
    p = os.path.abspath(path)
    ok = os.path.isdir(p)
    total, free, cnt = 0, 0, 0
    if ok:
        try:
            st = os.statvfs(p)
            LIMIT = 1 << 55
            for attr, key in (('f_blocks', 'total'), ('f_bavail', 'free')):
                v = getattr(st, attr, 0) * max(st.f_frsize or st.f_bsize or 0, 0)
                if v < 0 or v > LIMIT:
                    v = 0
                if key == 'total':
                    total = v
                else:
                    free = v
        except Exception:
            pass
        try:
            cnt = sum(1 for _ in scan_images(p, {'handle_jpg': True, 'handle_video': True}, limit=2000))
        except Exception:
            cnt = -1
    return {'path': p, 'ok': ok, 'removable': is_removable(p),
            'total_bytes': total, 'free_bytes': free, 'sample_count': cnt,
            'app': APP_NAME, 'version': APP_VER}

def path_inside(child, parent):
    if not child or not parent:
        return False
    c, p = os.path.realpath(os.path.abspath(child)), os.path.realpath(os.path.abspath(parent))
    return c == p or c.startswith(p + os.sep)

# ---- ExifTool 定位（三级：env → 随包 → 系统 PATH）----

_ET_BIN = None
_ET_PERL = None

def _et_candidates():
    base = Path(__file__).resolve().parent.parent
    cands = []
    envp = os.environ.get('EXIFTOOL_PATH')
    if envp:
        cands.append(envp)
    for rel in ('vendor/exiftool/exiftool', 'app/exiftool/exiftool',
                'thirdparty/exiftool/Image-ExifTool-13.00/exiftool'):
        cands.append(str(base / rel))
    for rel in ('exiftool', 'exiftool_files/exiftool'):
        cands.append(str(base.parent / rel))
```

```
    if sys.platform == 'win32':
        cands += ['exiftool.exe', str(base / 'vendor' / 'exiftool' / 'exiftool.exe')]
    cands.append('exiftool')
    return [c for c in cands if c]

def _et_run_prefix(binp):
    """返回调用前缀：.exe 直接执行，perl 脚本走 perl"""
    s = str(binp).lower()
    if s.endswith('.exe'):
        return [str(binp)]
    if os.path.basename(s) in ('exiftool',) and os.path.exists('/usr/bin/perl'):
        return ['perl', str(binp)]
    if os.path.basename(s) in ('exiftool',):
        return [str(binp)]
    return [str(binp)]

def _exiftool_bin():
    global _ET_BIN
    if _ET_BIN is not None:
        return _ET_BIN
    for c in _et_candidates():
        try:
            r = subprocess.run(_et_run_prefix(c) + ['-ver'], capture_output=True, text=True, timeout=8)
            if r.returncode == 0 and r.stdout.strip():
                _ET_BIN = c
                return c
        except Exception:
            continue
    _ET_BIN = False
    return False

def et_available():
    return bool(_exiftool_bin())

# ---- 日期解析（纯读，不动文件）----

def _parse_dt(v):
    if v is None:
        return None
    s = str(v).strip()
    for fmt in ('%Y:%m:%d %H:%M:%S', '%Y-%m-%d %H:%M:%S', '%Y:%m:%d', '%Y-%m-%d'):
        try:
            return datetime.strptime(s[:19], fmt)
        except Exception:
            pass
    try:
        n = float(s)
        if n > 1e9:
            return datetime.fromtimestamp(n)
    except Exception:
        pass
    return None

def _norm_make(m):
    if not m:
        return None
    s = str(m).strip()
    return BRAND_MAP.get(s.upper(), s)

def _get_make(path):
    if exifread:
        try:
            with open(path, 'rb') as f:
                t = exifread.process_file(f, details=False, stop_tag='Image Make')
            m = t.get('Image Make')
            if m:
                return _norm_make(m)
        except Exception:
            pass
    binp = _exiftool_bin()
    if binp:
        try:
            r = subprocess.run(_et_run_prefix(binp) + ['-j', '-Make', path],
                               capture_output=True, text=True, timeout=10)
            if r.returncode == 0 and r.stdout:
                o = json.loads(r.stdout)[0]
                return _norm_make(o.get('Make') or o.get('System:Make'))
        except Exception:
            pass
    return None
```

```
def resolve_date(path):
    """返回 (datetime|None, tag)。照片取不到 EXIF → (None,'unknown')；视频兜底 file_mtime。"""
    ext = os.path.splitext(path)[1].lower()
    video = ext in VID_EXTS

    if exifread and not video:
        try:
            with open(path, 'rb') as f:
                tags = exifread.process_file(f, details=False, stop_tag='DateTimeOriginal')
            for k in ('EXIF DateTimeOriginal', 'EXIF DateTimeDigitized', 'Image DateTime'):
                v = tags.get(k)
                if v:
                    dt = _parse_dt(v)
                    if dt:
                        return dt, 'exifread'
        except Exception:
            pass

    if Image and ext in JPG_EXTS:
        try:
            im = Image.open(path)
            im.load()
            ex = im._getexif()
            if isinstance(ex, dict):
                for k in (0x9003, 0x9004, 0x0132):
                    v = ex.get(k)
                    if v:
                        dt = _parse_dt(v)
                        if dt:
                            return dt, 'pillow'
        except Exception:
            pass

    binp = _exiftool_bin()
    if binp:
        try:
            et_tags = ['-CreateDate', '-DateTimeOriginal', '-MediaCreateDate', '-TrackCreateDate']
            et_keys = ('ExifIFD:CreateDate', 'ExifIFD:DateTimeOriginal',
                       'QuickTime:CreateDate', 'QuickTime:MediaCreateDate')
            if video:
                # 红线：文件修改时间兜底只允许用于视频，照片无 EXIF 视为 unknown 并跳过
                et_tags.append('-FileModifyDate')
                et_keys += ('System:FileModifyDate',)
            cmd = _et_run_prefix(binp) + ['-j', '-G1', '-n', '-m'] + et_tags + [path]
            r = subprocess.run(cmd, capture_output=True, text=True, timeout=30)
            if r.returncode == 0 and r.stdout:
                for o in json.loads(r.stdout):
                    for k in et_keys:
                        v = o.get(k)
                        if v:
                            dt = _parse_dt(v)
                            if dt:
                                return dt, k.split(':')[0].lower()
        except Exception:
            pass

    if video:
        try:
            return datetime.fromtimestamp(os.path.getmtime(path)), 'file_mtime'
        except Exception:
            pass
    return None, 'unknown'

# ---- 扫描 ----

def scan_images(root, opts, limit=None):
    root = os.path.abspath(root)
    cnt = 0
    for dn, dns, fns in os.walk(root, topdown=True, followlinks=False):
        dns[:] = [d for d in dns if d not in SKIP_DIRS and not DATE_DIR_RE.match(d)]
        dns.sort()
        for fn in sorted(fns):
            ext = os.path.splitext(fn)[1].lower()
            if ext not in ALL_EXTS:
                continue
            if ext in JPG_EXTS and not opts.get('handle_jpg', True):
                continue
            if ext in VID_EXTS and not opts.get('handle_video', True):
                continue
```

```
            yield os.path.join(dn, fn)
            cnt += 1
            if limit and cnt >= limit:
                return

# ---- 目标路径 ----

def build_target(rel_src, dt, opts, reserved, output):
    base_name = os.path.splitext(os.path.basename(rel_src))[0]
    ext = os.path.splitext(rel_src)[1].lower()
    date_str = dt.strftime('%Y%m%d') if dt else 'NoDate'
    ymd_dir = date_str if dt else '_NoDate'

    folder = ymd_dir
    if opts.get('brand_folder'):
        mk = opts.get('_maker_cache', {}).get(rel_src)
        if mk:
            folder = os.path.join(mk, ymd_dir)

    new_name = '%s_%s%s' % (base_name, date_str, ext)
    dst = os.path.join(output, folder, new_name)

    base, n = dst, 1
    while dst in reserved or os.path.exists(dst):
        dst = '%s_%d%s' % (os.path.splitext(base)[0], n, ext)
        n += 1
    reserved.add(dst)

    month_dst = None
    if opts.get('month_dir') and dt and ext in (JPG_EXTS | VID_EXTS):
        md = dt.strftime('%m')
        mpath = os.path.join(output, md, new_name)
        mb, mn = mpath, 1
        while mpath in reserved or os.path.exists(mpath):
            mpath = '%s_%d%s' % (os.path.splitext(mb)[0], mn, ext)
            mn += 1
        reserved.add(mpath)
        month_dst = mpath
    return dst, month_dst

# ---- 预览 ----

def do_preview(source, opts, limit=500):
    source = os.path.abspath(source)
    rows, maker_cache = [], {}
    by_type, bad_count, conflicts = defaultdict(int), 0, 0
    reserved = set()
    total_size = 0
    for fp in scan_images(source, opts):
        try:
            sz = os.path.getsize(fp)
        except Exception:
            continue
        total_size += sz
        ext = os.path.splitext(fp)[1].lower()
        by_type[ext] += 1
        rel = os.path.relpath(fp, source)
        dt, tag = resolve_date(fp)
        if tag in ('file_mtime', 'unknown'):
            bad_count += 1
        if opts.get('brand_folder'):
            maker_cache[rel] = _get_make(fp)
        opts['_maker_cache'] = maker_cache
        dst, _md = build_target(rel, dt, opts, reserved, source)
        # build_target 已就地消解重名（_1/_2），返回的 dst 必唯一；改用"无冲突目标名"判断本文件是否被加后缀
        clean_dst, _ = build_target(rel, dt, opts, set(), source)
        if clean_dst != dst:
            conflicts += 1
        rows.append({'src': rel, 'size': sz,
                     'date': dt.strftime('%Y-%m-%d') if dt else None,
                     'date_from': tag,
                     'dst_rel': os.path.relpath(dst, source) if dst else None,
                     'bad': tag in ('file_mtime', 'unknown'),
                     'skip': (ext in (RAW_EXTS | JPG_EXTS) and tag == 'unknown')})
        if limit and len(rows) >= limit:
            break
    dates = sorted({r['date'] for r in rows if r['date']})
    return {'source': source, 'removable': is_removable(source),
            'total': len(rows), 'size': total_size, 'by_type': dict(by_type),
            'date_range': [dates[0], dates[-1]] if dates else [None, None],
```

```
            'bad_count': bad_count, 'conflicts': conflicts,
            'raw_skip_hint': sum(1 for r in rows if r['skip']),
            'removable_note': 'move 模式归档成功后将清空源设备' if is_removable(source) else '',
            'exiftool': et_available(),
            'sample': rows[:limit]}

# ---- 任务态 ----

def task_dir(task_id, output):
    if output:
        return os.path.join(output, APP_DIR, 'tasks', task_id)
    base = Path(__file__).resolve().parent.parent / 'data' / 'rawarchiver' / 'tasks'
    return str(base / task_id)

def load_state(tid, output):
    p = Path(task_dir(tid, output)) / 'state.json'
    return json.loads(p.read_text(encoding='utf-8')) if p.exists() else None

def cancelled(tid, output):
    return (Path(task_dir(tid, output)) / 'cancel').exists()

# ---- 内部暂存 ----

def tree_copy(src, dst, log_cb=None):
    os.makedirs(dst, exist_ok=True)
    n, b = 0, 0
    for fp in scan_images(src, {'handle_jpg': True, 'handle_video': True}):
        rel = os.path.relpath(fp, src)
        tp = os.path.join(dst, rel)
        os.makedirs(os.path.dirname(tp), exist_ok=True)
        shutil.copy2(fp, tp)
        n += 1
        b += os.path.getsize(tp)
        if log_cb and n % 50 == 0:
            log_cb('  暂存 %d 个文件 / %d MB' % (n, b >> 20))
    return n, b

# ---- 清源 ----

def do_cleanup(output, task_id, verify_hash=False):
    td = task_dir(task_id, output)
    sp = Path(td) / 'state.json'
    if not sp.exists():
        return {'ok': False, 'error': 'state.json 不存在，无法定位源'}
    st = json.loads(sp.read_text(encoding='utf-8'))
    src = st.get('source')
    if not src or not os.path.isdir(src):
        return {'ok': False, 'error': '源不可访问: %s（可能已拔卡或挂载点变化）' % src}
    ok, kept, fail = 0, [], []
    for it in st.get('to_delete', []):
        p = os.path.join(src, it['src'])
        try:
            if not os.path.isfile(p):
                ok += 1
                continue
            if os.path.getsize(p) != it['size']:
                kept.append(it['src'])
                continue
            if verify_hash and it.get('md5'):
                h = hashlib.md5()
                with open(p, 'rb') as f:
                    for chunk in iter(lambda: f.read(1 << 16), b''):
                        h.update(chunk)
                if h.hexdigest() != it['md5']:
                    kept.append(it['src'])
                    continue
            os.unlink(p)
            ok += 1
        except Exception as e:
            fail.append('%s: %s' % (it['src'], e))
    for root, dirs, _ in os.walk(src, topdown=False):
        for d in dirs:
            dp = os.path.join(root, d)
            try:
                if d.lower() != '@eadir' and not os.listdir(dp):
                    os.rmdir(dp)
            except Exception:
                pass
    return {'ok': True, 'deleted': ok, 'kept': kept, 'fail': fail}
```

```
# ---- 回滚（仅 copy 模式 + 本地源有效）----

def do_rollback(output, task_id):
    td = task_dir(task_id, output)
    sp = Path(td) / 'state.json'
    if not sp.exists():
        return {'ok': False, 'error': 'state.json 不存在'}
    st = json.loads(sp.read_text(encoding='utf-8'))
    if st.get('mode') == 'move' and st.get('removable'):
        return {'ok': False, 'error': '可移动介质已清源，无法回滚，请依靠 _report_*.json 清单'}
    moved, n = os.path.join(output, '_restored', st.get('id', task_id)), 0
    for it in st.get('done', []):
        if it.get('status') != 'ok':
            continue
        d = os.path.join(output, it['dst'])
        if not os.path.isfile(d):
            continue
        tp = os.path.join(moved, it['dst'])
        os.makedirs(os.path.dirname(tp), exist_ok=True)
        shutil.move(d, tp)
        n += 1
    st['status'] = 'rolled_back'
    atomic_write(sp, st)
    return {'ok': True, 'restored': n, 'to': moved}

# ---- 主流程 ----

def run_task(source, output, opts, task_id):
    source = os.path.abspath(source)
    output = os.path.abspath(output)
    removable = is_removable(source)
    mode = opts.get('mode') or 'copy'
    opts['mode'] = mode

    td = task_dir(task_id, output)
    os.makedirs(td, exist_ok=True)
    sp = Path(td) / 'state.json'

    st = load_state(task_id, output)
    if not st:
        st = {'id': task_id, 'app': APP_NAME, 'version': APP_VER,
              'source': source, 'output': output, 'mode': mode, 'removable': removable,
              'brand_folder': bool(opts.get('brand_folder')),
              'handle_jpg': bool(opts.get('handle_jpg', True)),
              'handle_video': bool(opts.get('handle_video', True)),
              'month_dir': bool(opts.get('month_dir')), 'staging': bool(opts.get('staging', True)),
              'total': 0, 'current': 0, 'success': 0, 'failed': 0, 'skipped': 0,
              'status': 'pending', 'error': '', 'logs': [], 'done': [], 'to_delete': [],
              'created_at': now(), 'updated_at': now(), 'finished_at': None}
        atomic_write(sp, st)
    else:
        st['status'] = 'resuming'
        atomic_write(sp, st)

    def save():
        st['updated_at'] = now()
        atomic_write(sp, st)

    stage = None
    try:
        src = source
        if opts.get('staging', True) and removable:
            stage = os.path.join(td, 'stage')
            log_msg(st, '[暂存] 开始顺序复制到 NAS ...')
            t0 = now()
            n, b = tree_copy(src, stage, lambda m: log_msg(st, m))
            log_msg(st, '[暂存] 完成 %d 个文件 / %d MB，耗时 %ds' % (n, b >> 20, int(now() - t0)))
            src = stage

        log_msg(st, '模式: %s | 源为可移动介质: %s | 暂存: %s | ExifTool: %s'
                % (mode, removable, bool(stage), et_available()))
        st['status'] = 'running'
        save()

        reserved = set()
        done_key = {(it['src'], it['size']) for it in st.get('done', [])}
        st['skipped'] = 0
        st['failed'] = 0
        items = list(scan_images(src, opts))
        st['total'] = len(items)
```

```
save()
log_msg(st, '待处理 %d 个文件' % len(items))

maker_cache = {}
if opts.get('brand_folder'):
    log_msg(st, '正在识别品牌（仅本次）...')
    for fp in items:
        maker_cache[os.path.relpath(fp, src)] = _get_make(fp)
opts['_maker_cache'] = maker_cache

for idx, fp in enumerate(items, 1):
    if cancelled(task_id, output):
        st['status'] = 'cancelled'
        log_msg(st, '已取消')
        break
    st['current'] = idx
    save()
    rel = os.path.relpath(fp, src)
    ext = os.path.splitext(fp)[1].lower()
    try:
        sz = os.path.getsize(fp)
    except Exception:
        sz = 0
    key = (rel, sz)
    if key in done_key:
        st['skipped'] += 1
        continue
    dt, tag = resolve_date(fp)
    if dt is None:
        st['skipped'] += 1
        done_key.add(key)
        st['done'].append({'src': rel, 'size': sz, 'status': 'skip', 'reason': 'no_date'})
        st['logs'].append({'t': datetime.now().strftime('%H:%M:%S'),
                           'status': 'skip', 'src': rel, 'msg': '无拍摄日期，跳过'})
        save()
        continue
    dst, mdst = build_target(rel, dt, opts, reserved, output)
    try:
        os.makedirs(os.path.dirname(dst), exist_ok=True)
        if mode == 'move' and not removable and not stage:
            shutil.move(fp, dst)
        else:
            shutil.copy2(fp, dst)
        if mdst:
            os.makedirs(os.path.dirname(mdst), exist_ok=True)
            shutil.copy2(dst, mdst)
        st['success'] += 1
        done_key.add(key)
        ent = {'src': rel, 'size': sz, 'dst': os.path.relpath(dst, output),
               'date': dt.strftime('%Y-%m-%d'), 'date_from': tag, 'status': 'ok'}
        if mdst:
            ent['month_dst'] = os.path.relpath(mdst, output)
        st['done'].append(ent)
        if mode == 'move':
            st['to_delete'].append({'src': rel, 'size': sz})
        st['logs'].append({'t': datetime.now().strftime('%H:%M:%S'), 'status': 'ok',
                           'src': rel, 'dst': ent['dst'], 'date': ent['date'], 'from': tag})
    except Exception as e:
        st['failed'] += 1
        st['logs'].append({'t': datetime.now().strftime('%H:%M:%S'),
                           'status': 'fail', 'src': rel, 'msg': str(e)})
    save()

if mode == 'move' and removable and st['to_delete']:
    log_msg(st, '[清源] 归档成功，开始清空源设备上对应文件 ...')
    r = do_cleanup(output, task_id)
    if r.get('ok'):
        msg = '[清源] 已删除 %d 个文件' % r['deleted']
        if r['kept']:
            msg += ', %d 个因大小校验未过而保留' % len(r['kept'])
        log_msg(st, msg)
        if r['fail']:
            log_msg(st, '[清源] 失败: ' + '; '.join(r['fail'][:5]))
    else:
        log_msg(st, '[清源] 跳过（文件已安全归档）: %s, 后续可用 --cleanup 补清' % r.get('error'))

if st['status'] != 'cancelled':
    st['status'] = 'done'
st['finished_at'] = now()
log_msg(st, '完成: 成功 %d / 失败 %d / 跳过 %d' % (st['success'], st['failed'], st['skipped']))
```

```
        save()
        write_report(output, task_id, st)
    except Exception as e:
        st['status'] = 'error'
        st['error'] = str(e)
        st['logs'].append({'t': datetime.now().strftime('%H:%M:%S'),
                           'status': 'fail', 'msg': '异常: %s' % e})
        st['finished_at'] = now()
        save()
        raise
    finally:
        if stage and os.path.isdir(stage):
            shutil.rmtree(stage, ignore_errors=True)
            log_msg(st, '[暂存] 临时目录已清理')
            save()
    return st

def write_report(output, task_id, st):
    rp = os.path.join(output, '_report_%s.json' % task_id)
    atomic_write(rp, {
        'app': APP_NAME, 'version': APP_VER,
        'task_id': task_id, 'source': st['source'], 'output': st['output'],
        'removable': st['removable'], 'mode': st['mode'],
        'total': st['total'], 'success': st['success'],
        'failed': st['failed'], 'skipped': st['skipped'],
        'items': [d for d in st.get('done', []) if d.get('status') == 'ok'],
        'finished_at': datetime.fromtimestamp(st['finished_at']).isoformat() if st.get('finished_at') else None,
    })

# ---- CLI ----

def main():
    ap = argparse.ArgumentParser(description='%s v%s' % (APP_NAME, APP_VER))
    ap.add_argument('source', nargs='?', help='源目录（卡/U盘/本地目录）')
    ap.add_argument('--output', '-o', help='输出目录（归档落点）')
    ap.add_argument('--task-id', default=None, help='任务ID（续跑用）')
    ap.add_argument('--mode', choices=['copy', 'move'], default='copy', help='copy=留源, move=归档后清源')
    ap.add_argument('--preview', action='store_true', help='只预览不写盘, 输出 JSON')
    ap.add_argument('--info', action='store_true', help='打印源信息 JSON')
    ap.add_argument('--doctor', action='store_true', help='自检依赖与 ExifTool')
    ap.add_argument('--cleanup', action='store_true', help='补清源')
    ap.add_argument('--rollback', action='store_true', help='回滚（仅 copy 模式）')
    ap.add_argument('--verify-hash', action='store_true', help='清源时做 MD5 校验（慢）')
    ap.add_argument('--staging', dest='staging', action='store_true', default=True)
    ap.add_argument('--no-staging', dest='staging', action='store_false')
    ap.add_argument('--brand', dest='brand_folder', action='store_true')
    ap.add_argument('--no-jpg', dest='handle_jpg', action='store_false', default=True)
    ap.add_argument('--no-video', dest='handle_video', action='store_false', default=True)
    ap.add_argument('--month', dest='month_dir', action='store_true')
    args = ap.parse_args()

    opts = {'brand_folder': args.brand_folder, 'handle_jpg': args.handle_jpg,
            'handle_video': args.handle_video, 'month_dir': args.month_dir,
            'staging': args.staging, 'mode': args.mode}

    if args.doctor:
        print(json.dumps({'app': APP_NAME, 'version': APP_VER,
                          'python': sys.version.split()[0],
                          'exifread': bool(exifread), 'pillow': bool(Image),
                          'exiftool': _exiftool_bin() or None,
                          'perl': os.path.exists('/usr/bin/perl')}, ensure_ascii=False))
        return 0
    if args.info:
        print(json.dumps(source_info(args.source), ensure_ascii=False))
        return 0
    if args.rollback:
        if not args.output or not args.task_id:
            print(json.dumps({'ok': False, 'error': '--rollback 需 --output 与 --task-id'}), file=sys.stderr)
            return 1
        print(json.dumps(do_rollback(args.output, args.task_id), ensure_ascii=False))
        return 0
    if args.preview:
        out = args.output or '/tmp/preview'
        print(json.dumps(do_preview(args.source, opts), ensure_ascii=False))
        return 0
    if args.cleanup:
        if not args.output or not args.task_id:
            print(json.dumps({'ok': False, 'error': '--cleanup 需 --output 与 --task-id'}), file=sys.stderr)
            return 1
        print(json.dumps(do_cleanup(args.output, args.task_id, args.verify_hash), ensure_ascii=False))
```

```
        return 0
    if not args.source:
        print(json.dumps({'ok': False, 'error': '缺少 source'}), file=sys.stderr)
        return 1
    if not args.output:
        print(json.dumps({'ok': False, 'error': '--output 必填'}), file=sys.stderr)
        return 1
    if path_inside(args.output, args.source):
        print(json.dumps({'ok': False, 'error': '输出目录不能位于源目录内部'}), file=sys.stderr)
        return 1
    if not os.path.isdir(args.source):
        print(json.dumps({'ok': False, 'error': '源不存在: %s' % args.source}), file=sys.stderr)
        return 1
    try:
        os.makedirs(args.output, exist_ok=True)
        testf = os.path.join(args.output, '.write_test')
        open(testf, 'w').close()
        os.unlink(testf)
    except Exception as e:
        print(json.dumps({'ok': False, 'error': '输出目录不可写: %s' % e}), file=sys.stderr)
        return 1

    tid = args.task_id or time.strftime('%Y%m%d_%H%M%S') + '_%d' % os.getpid()
    st = run_task(args.source, args.output, opts, tid)
    print(json.dumps({'ok': st['status'] in ('done', 'cancelled'), 'taskId': tid,
                      'status': st['status'], 'success': st['success'],
                      'failed': st['failed'], 'skipped': st['skipped']}, ensure_ascii=False))
    return 0 if st['status'] == 'done' else 1

if __name__ == '__main__':
    sys.exit(main())
```

app/rawarchiver/www/index.html（Web 面板，完整版）

```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
<meta charset="utf-8"><meta name="viewport" content="width=device-width,initial-scale=1">
<meta name="description" content="RAW 归档 —— 相机照片按拍摄日期整理归档">
<title>RAW 归档</title>
<style>
body{font-family:-apple-system,"Microsoft YaHei",sans-serif;max-width:980px;margin:0 auto;padding:20px;color:#222}
h1 small{font-size:14px;color:#666;font-weight:400}
label{display:block;margin-top:14px;font-weight:600}
input[type=text],select{width:100%;padding:7px;margin-top:4px;box-sizing:border-box}
input[type=checkbox]{margin-right:6px}
button{margin-top:12px;padding:8px 16px;cursor:pointer;border:1px solid #bbb;background:#f5f5f5;border-radius:4px}
button:disabled{opacity:.45;cursor:not-allowed}
.row{display:flex;gap:8px;align-items:flex-end}
.row>*{flex:1}.row>button{flex:0 0 auto;margin-top:4px}
table{width:100%;border-collapse:collapse;margin-top:10px;font-size:13px}
th,td{border:1px solid #ddd;padding:5px;text-align:left;word-break:break-all}
.bad{color:#c00;font-weight:600}.warn{color:#c00;margin-top:8px;line-height:1.6}
.hint{color:#080;font-size:13px;margin-left:8px}
#logBox{max-height:240px;overflow:auto;font-family:monospace;font-size:12px;background:#f7f7f7;padding:8px;margin-top:8px;white-space:pre-wrap}
#progBar{height:18px;background:#eee;border-radius:4px;overflow:hidden;margin-top:10px}
#progFill{width:0;height:100%;background:#4a9;transition:width .3s}
footer{margin-top:28px;color:#999;font-size:12px;border-top:1px solid #eee;padding-top:10px}
</style>
</head>
<body>
<h1>RAW 归档 <small>v1.0.0</small></h1>

<label>源（SD/TF/U盘或本地目录）
  <div class="row" style="margin-top:4px">
    <select id="srcSel"><option value="">— 请选择已挂载设备 —</option></select>
    <button type="button" id="refreshSrc">刷新设备</button>
  </div>
  <input type="text" id="srcPath" placeholder="或手填路径，如 /volumeUSB1/usbshare">
</label>

<label>输出目录（归档落点，不存在会自动创建）
  <div class="row" style="margin-top:4px">
    <input type="text" id="outPath" placeholder="/volume1/photos/Camera">
    <button type="button" id="mkOut">创建目录</button>
  </div>
  <span id="outHint" class="hint"></span>
</label>
```

```
<label>选项</label>
<div style="line-height:2">
<label style="display:inline-block;margin:0 16px 0 0;font-weight:400"><input type="checkbox" id="optBrand"> 按品牌分层</label>
<label style="display:inline-block;margin:0 16px 0 0;font-weight:400"><input type="checkbox" id="optJpg" checked> 处理 JPG</label>
<label style="display:inline-block;margin:0 16px 0 0;font-weight:400"><input type="checkbox" id="optVideo" checked> 处理视频</label>
<label style="display:inline-block;margin:0 16px 0 0;font-weight:400"><input type="checkbox" id="optMonth" checked> 月份平铺副本</label>
<label style="display:inline-block;margin:0 16px 0 0;font-weight:400"><input type="checkbox" id="optStaging" checked> 内部暂存（卡源推荐）</labe
l>
</div>

<label>模式
  <select id="optMode">
    <option value="copy">复制（保留源文件）</option>
    <option value="move">移动（归档后清空源设备，不可回滚）</option>
  </select>
</label>

<div>
  <button type="button" id="btnPreview">预览</button>
  <button type="button" id="btnStart" disabled>确认并开始</button>
  <button type="button" id="btnCancel" disabled>取消</button>
  <button type="button" id="btnCleanup">补清源</button>
  <button type="button" id="btnRollback">回滚</button>
</div>

<div id="previewBox" style="display:none;margin-top:16px">
  <div id="prevSummary"></div>
  <div id="prevWarn" class="warn"></div>
  <table><thead><tr><th>源文件</th><th>拍摄日期</th><th>日期来源</th><th>目标路径</th></tr></thead>
  <tbody id="prevBody"></tbody></table>
</div>

<div id="statusBox" style="margin-top:16px;display:none">
  <div id="progBar"><div id="progFill"></div></div>
  <div id="progText"></div>
  <div id="logBox"></div>
</div>

<footer>RAW 归档 · 源设备只读优先，移动模式仅在归档全部成功后清源 · 任务态写在 &lt;输出目录&gt;/.rawarchiver/</footer>

<script>
const API = (location.pathname.replace(/\/(www\/)?(index\.html)?$/, '') || '/app/rawarchiver') + '/api/index.cgi';
const LS = 'rawarchiver.state';
const $ = id => document.getElementById(id);
const api = (act, body={}) => fetch(API + '?action=' + act, {
  method:'POST', headers:{'Content-Type':'application/json'},
  body: JSON.stringify(Object.assign({action:act}, body))
}).then(r => r.json()).catch(e => ({ok:false, error:'请求失败: ' + e}));

let tid = null, timer = null;
const save = () => localStorage.setItem(LS, JSON.stringify({src:$('srcPath').value, out:$('outPath').value, mode:$('optMode').value}));
const load = () => { try { const o = JSON.parse(localStorage.getItem(LS)||'{}');
  $('srcPath').value = o.src||''; $('outPath').value = o.out||''; if (o.mode) $('optMode').value = o.mode; } catch(e){} };
load();

const isRemovable = () => /^\/(volumeUSB|mnt\/usb|media|run\/media|Volumes)/.test($('srcPath').value.trim());
const check = () => { const ok = $('srcPath').value.trim() && $('outPath').value.trim() && $('previewBox').dataset.ready === '1';
  $('btnStart').disabled = !ok; };
$('srcPath').oninput = $('outPath').oninput = () => { save(); check(); $('outHint').textContent=''; };
$('optMode').onchange = save;

$('refreshSrc').onclick = async () => {
  const r = await api('devices');
  const sel = $('srcSel');
  sel.innerHTML = '<option value="">— 请选择已挂载设备 —</option>';
  (r.list||[]).forEach(d => { const o = document.createElement('option');
    o.value = d.path; o.textContent = (d.label||'未命名卷') + ' (' + d.path + ') ' + d.files + ' 个可归档文件';
    sel.appendChild(o); });
  if (!(r.list||[]).length) sel.innerHTML = '<option value="">未发现 /volumeUSB* 设备，可手填路径</option>';
  sel.onchange = () => { $('srcPath').value = sel.value; save(); check(); };
};
$('refreshSrc').click();

$('mkOut').onclick = async () => {
  const r = await api('mkdir', {path: $('outPath').value.trim()});
  $('outHint').textContent = r.ok ? '已创建/已存在' : (r.error || '创建失败');
  $('outHint').style.color = r.ok ? '#080' : '#c00';
};

$('btnPreview').onclick = async () => {
```

```
  const body = {source: $('srcPath').value.trim(), output: $('outPath').value.trim(),
    brandFolder: $('optBrand').checked, handleJpg: $('optJpg').checked,
    handleVideo: $('optVideo').checked, monthDir: $('optMonth').checked, staging: $('optStaging').checked};
  if (!body.source || !body.output) { alert('请先填写源与输出目录'); return; }
  $('prevSummary').textContent = '正在扫描源（读卡较慢，请稍候）...';
  $('previewBox').style.display = 'block'; $('previewBox').dataset.ready = '';
  const r = await api('preview', body);
  if (!r.ok) { $('prevSummary').textContent=''; $('prevWarn').textContent = r.error; return; }
  const d = r.result;
  $('prevSummary').innerHTML = '<b>' + d.total + '</b> 个文件 / ' + (d.size/1073741824).toFixed(2)
    + ' GB | 日期 ' + (d.date_range[0]||'?') + ' ~ ' + (d.date_range[1]||'?')
    + ' | ExifTool: ' + (d.exiftool ? '可用' : '未找到（视频日期会退化）');
  let w = '';
  if (d.bad_count) w += '注意：' + d.bad_count + ' 个文件只能靠文件修改时间推断日期，请核对后再执行。<br>';
  if (d.raw_skip_hint) w += '提示：' + d.raw_skip_hint + ' 个照片读不到拍摄日期，执行时将被跳过（不拷贝）。<br>';
  if (d.conflicts) w += '提示：' + d.conflicts + ' 个目标已存在同名文件，将自动追加 _1/_2。<br>';
  if (d.removable && $('optMode').value === 'move')
    w += '<b>源为可移动设备，移动模式下归档完成后将清空卡上约 ' + (d.size/1073741824).toFixed(2)
       + ' GB 文件，此操作不可回滚。</b>';
  $('prevWarn').innerHTML = w;
  $('prevBody').innerHTML = (d.sample||[]).map(s => '<tr><td>' + s.src + '</td><td>' + (s.date||'-')
    + '</td><td class="' + (s.bad?'bad':'') + '">' + s.date_from + '</td><td>' + (s.dst_rel||'') + '</td></tr>').join('');
  $('previewBox').dataset.ready = '1'; check();
};

$('btnStart').onclick = async () => {
  if ($('optMode').value === 'move' && isRemovable() &&
      !confirm('确认归档后清空源设备上的文件？此操作不可回滚。')) return;
  const r = await api('start', {source: $('srcPath').value.trim(), output: $('outPath').value.trim(),
    mode: $('optMode').value, brandFolder: $('optBrand').checked, handleJpg: $('optJpg').checked,
    handleVideo: $('optVideo').checked, monthDir: $('optMonth').checked, staging: $('optStaging').checked});
  if (!r.ok) { alert(r.error); return; }
  tid = r.taskId; $('btnStart').disabled = true; $('btnCancel').disabled = false;
  $('statusBox').style.display = 'block'; $('logBox').textContent = '';
  timer = setInterval(poll, 1500); poll();
};

async function poll(){
  const r = await api('status', {id: tid, output: $('outPath').value.trim()});
  if (!r.ok) return;
  const s = r.state;
  const pct = s.total ? Math.round(s.current/s.total*100) : 0;
  $('progFill').style.width = pct + '%';
  $('progText').textContent = s.status + '  ' + s.current + '/' + s.total
    + ' | 成功 ' + s.success + ' 失败 ' + s.failed + ' 跳过 ' + s.skipped;
  $('logBox').textContent = (s.logs||[]).slice(-80).map(l => '[' + l.t + '] ' + (l.msg || l.dst || l.src)).join('\n');
  $('logBox').scrollTop = $('logBox').scrollHeight;
  if (['done','error','cancelled','rolled_back'].indexOf(s.status) >= 0) {
    clearInterval(timer); $('btnCancel').disabled = true; $('btnStart').disabled = false;
    if (s.status === 'done') alert('RAW 归档完成：成功 ' + s.success + ' 个');
  }
}

$('btnCancel').onclick = async () => { const r = await api('cancel', {taskId: tid, output: $('outPath').value.trim()});
  if (!r.ok) alert(r.error); };
$('btnCleanup').onclick = async () => {
  if (!tid) tid = prompt('输入任务ID（输出目录 .rawarchiver/tasks/ 下的目录名）');
  if (!tid) return;
  const r = await api('cleanup', {taskId: tid, output: $('outPath').value.trim()});
  alert(r.ok ? ('已删除 ' + r.deleted + ' 个' + (r.kept && r.kept.length ? (', 保留 ' + r.kept.length + ' 个（大小校验未过）') : '')) : r.error)
;
};
$('btnRollback').onclick = async () => {
  if ($('optMode').value === 'move') { alert('移动模式下源文件已清除，无法回滚，请依靠 _report_*.json 清单处理。'); return; }
  if (!tid) { alert('请先执行或输入任务ID'); return; }
  if (!confirm('确认把本任务已归档的文件移回输出目录下的 _restored/ ？')) return;
  const r = await api('rollback', {taskId: tid, output: $('outPath').value.trim()});
  alert(r.ok ? ('已还原 ' + r.restored + ' 个到 ' + r.to) : r.error);
};
</script>
</body>
</html>
```

app/rawarchiver/api/index.cgi（DSM CGI 入口）

```
#!/usr/bin/perl
# RAW 归档 v1.0.0 —— DSM CGI 入口
use strict; use warnings; use utf8;
use CGI; use JSON::PP; use Cwd qw(realpath);
```

```
binmode STDOUT, ':utf8';

my $q = CGI->new;
my $APP_NAME = 'RAW 归档';
my $APP_VER  = '1.0.0';

# 三个路径由 installer.sh 写入 env 文件；也允许环境变量覆盖
my $HOME_DIR = $ENV{RAWARCHIVER_HOME} || '/volume1/web/rawarchiver';
if (open my $ef, '<', "$HOME_DIR/etc/env") { my $l = <$ef>; close $ef; chomp $l; $HOME_DIR = $l if $l; }
my $PY     = $ENV{RAWARCHIVER_PY}    || "$HOME_DIR/agent/venv/bin/python3";
$PY        = '/usr/bin/python3' unless -x $PY;
my $SCRIPT = "$HOME_DIR/scripts/raw_archiver.py";
my $DATA   = "$HOME_DIR/data/rawarchiver";

print $q->header(-type => 'application/json; charset=UTF-8');

sub err { print encode_json({ok=>0, error=>$_[0]}); exit 0; }
sub okj { print encode_json({ok=>1, @_}); exit 0; }

sub safe {
    my $p = shift // '';
    $p =~ s{^/+}{/};
    err("路径不合法：必须以 /volume 开头且不含 ..") unless $p =~ m{^/volume} && $p !~ /\.\./;
    return $p;
}
sub inside {
    my ($c, $p) = @_;
    return 0 unless $c && $p;
    $c = realpath($c) // ''; $p = realpath($p) // '';
    return ($c eq $p || $c =~ m{^\Q$p\E/}) ? 1 : 0;
}
sub task_dir { my ($id, $out) = @_; $out ? "$out/.rawarchiver/tasks/$id" : "$DATA/tasks/$id" }
sub load_state {
    my ($id, $out) = @_;
    my $sp = task_dir($id, $out) . '/state.json';
    return undef unless -f $sp;
    open my $fh, '<:utf8', $sp or return undef; my $s = do { local $/; <$fh> }; close $fh;
    my $j = eval { decode_json($s) }; return $j;
}
sub run_py {
    my @a = ($PY, '-X', 'utf8', $SCRIPT, @_);
    my $out = `@a 2>&1`;
    return $out;
}
sub jargs {
    my $p = shift; my @a;
    push @a, '--brand'     if $p->{brandFolder};
    push @a, '--no-jpg'    unless !defined $p->{handleJpg}  || $p->{handleJpg};
    push @a, '--no-video'  unless !defined $p->{handleVideo}|| $p->{handleVideo};
    push @a, '--month'     if $p->{monthDir};
    push @a, ($p->{staging} // 1) ? '--staging' : '--no-staging';
    return @a;
}

my $body = do { local $/; <STDIN> };
my $p = ($body && length $body) ? (eval { decode_json($body) } || {}) : {};
my $act = $p->{action} || $q->param('action') || 'info';

if ($act eq 'info') {
    okj(app=>$APP_NAME, version=>$APP_VER, script=>$SCRIPT, python=>$PY,
        exiftool=>(-f "$HOME_DIR/vendor/exiftool/exiftool" ? 1 : 0));
}
elsif ($act eq 'doctor') {
    okj(result=> eval { decode_json(run_py('--doctor')) } || {});
}
elsif ($act eq 'devices') {
    my @list;
    for my $d (glob('/volumeUSB*/usbshare* /volumeUSB*/usbshare')) {
        next unless -d $d;
        my $lab = `synosetkeyvalue 2>/dev/null`;   # 占位，卷标读不到则留空
        my $n = `find "$d" -maxdepth 5 -type f \\( -iname '*.nef' -o -iname '*.cr2' -o -iname '*.cr3' -o -iname '*.arw' -o -iname '*.raf' -o -in
ame '*.orf' -o -iname '*.dng' -o -iname '*.jpg' -o -iname '*.jpeg' -o -iname '*.mp4' -o -iname '*.mov' -o -iname '*.mts' \\) 2>/dev/null | wc -l
`;
        chomp $n;
        my ($vol) = $d =~ m{^(/volumeUSB\d+)};
        my $sz = `df -k "$vol" 2>/dev/null | awk 'NR==2{print \$2}'`; chomp $sz;
        push @list, {path=>$d, label=>'', files=>$n + 0, total_kb=>($sz // 0) + 0};
    }
    okj(list=>\@list);
}
```

```
elsif ($act eq 'mkdir') {
    my $path = safe($p->{path});
    mkpath_of($path);
    okj(path=>$path);
}
elsif ($act eq 'preview') {
    err('source 必填') unless $p->{source};
    my $out = safe($p->{output});
    my $r = run_py($p->{source}, '--output', $out, '--preview', jargs($p));
    my $j = eval { decode_json($r) };
    err("预览失败: " . substr($r, 0, 800)) unless $j;
    okj(result=>$j);
}
elsif ($act eq 'start') {
    err('source 与 output 必填') unless $p->{source} && $p->{output};
    my $out = safe($p->{output});
    err('输出目录不能位于源目录内部') if inside($out, $p->{source});
    err('源目录不存在') unless -d $p->{source};
    mkpath_of("$out/.rawarchiver/tasks");
    my $tid = $p->{taskId} || do { my @t = localtime(); sprintf('%04d%02d%02d_%02d%02d%02d', $t[5]+1900, $t[4]+1, $t[3], $t[2], $t[1], $t[0]) .
'_' . int(rand(900)+100) };
    my @a = ($PY, '-X', 'utf8', $SCRIPT, $p->{source}, '--output', $out,
             '--mode', ($p->{mode} && $p->{mode} eq 'move' ? 'move' : 'copy'),
             '--task-id', $tid, jargs($p));
    my $cmd = join(' ', map { "'$_'" } @a);
    # 后台分离执行，CGI 立即返回
    system("/bin/sh -c \"nohup $cmd > '$out/.rawarchiver/tasks/$tid/run.log' 2>&1 &\"");
    okj(taskId=>$tid);
}
elsif ($act eq 'status') {
    err('id 必填') unless $p->{id};
    my $st = load_state($p->{id}, $p->{output} ? safe($p->{output}) : '');
    err('任务不存在') unless $st;
    $st->{logs} = [ @{$st->{logs} || []} ];
    $st->{logs} = [ @{$st->{logs}}[ -80 .. -1 ] ] if @{$st->{logs}} > 80;
    delete $st->{done}; delete $st->{to_delete};
    okj(state=>$st);
}
elsif ($act eq 'cancel') {
    my $td = task_dir($p->{taskId}, $p->{output} ? safe($p->{output}) : '');
    err('任务不存在') unless -d $td;
    open my $fh, '>', "$td/cancel"; close $fh;
    okj();
}
elsif ($act eq 'cleanup') {
    my $out = safe($p->{output});
    err('taskId 必填') unless $p->{taskId};
    my $r = run_py('--output', $out, '--task-id', $p->{taskId}, '--cleanup');
    my $j = eval { decode_json($r) };
    okj($j ? %$j : (ok=>0, error=>substr($r, 0, 800)));
}
elsif ($act eq 'rollback') {
    my $out = safe($p->{output});
    err('taskId 必填') unless $p->{taskId};
    my $st = load_state($p->{taskId}, $out);
    err('移动模式不可回滚，请依靠 _report_*.json 清单') if $st && ($st->{mode} // '') eq 'move';
    my $r = run_py('--output', $out, '--task-id', $p->{taskId}, '--rollback');
    my $j = eval { decode_json($r) };
    okj($j ? %$j : (ok=>0, error=>substr($r, 0, 800)));
}
else { err("未知动作: $act") }

sub mkpath_of {
    my $path = shift;
    return if -d $path;
    my @parts = grep { length } split m{/}, $path;
    my $cur = '';
    for my $x (@parts) { $cur .= "/$x"; mkdir $cur unless -d $cur; }
}
```

agent/agent.py（本机 FastAPI 版）

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
RAW 归档 —— 本机版 Agent（FastAPI）
用途：NAS CGI 以 http 身份运行、删卡上文件可能没权限时，用本机/root 身份跑同一套核心脚本。
启动：python agent/agent.py   然后浏览器打开 http://127.0.0.1:8765/
"""
```

```
import os, sys, json, time, random, shutil, threading, subprocess, webbrowser
from pathlib import Path
from fastapi import FastAPI, Request
from fastapi.middleware.cors import CORSMiddleware
from fastapi.staticfiles import StaticFiles
import uvicorn

ROOT = Path(__file__).resolve().parent.parent
SCRIPT = ROOT / "scripts" / "raw_archiver.py"
APP_NAME = "RAW Archiver"
APP_VER = "1.0.0"

# Web 静态目录在 NAS 布局下是 app/rawarchiver/www, 本机同样可用
WWW = ROOT / "app" / "rawarchiver" / "www"

def py():
    venv = ROOT / "agent" / "venv" / ("Scripts/python.exe" if sys.platform == "win32" else "bin/python")
    return str(venv) if venv.exists() else sys.executable

app = FastAPI(title=APP_NAME)
app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["*"], allow_headers=["*"])

def call(*args, timeout=1800):
    env = dict(os.environ)
    env.setdefault("PYTHONIOENCODING", "utf-8")
    return subprocess.check_output([py(), "-X", "utf8", str(SCRIPT), *[str(a) for a in args]],
                                   text=True, timeout=timeout, env=env, cwd=str(ROOT))

def list_devices():
    out = []
    if sys.platform == "win32":
        for lit in "DEFGHIJKLMNOPQRSTUVWXYZ":
            root = f"{lit}:\\"
            try:
                if Path(root).exists():
                    n = sum(1 for _ in Path(root).rglob("*.JPG"))
                    out.append({"path": root, "label": f"盘 {lit}", "files": n})
            except Exception:
                pass
    else:
        for pat in ("/volumeUSB*/usbshare*", "/media/*/*", "/run/media/*/*", "/mnt/usb*"):
            for d in sorted(ROOT.glob("/*")) if False else sorted(Path("/").glob(pat.lstrip("/"))):
                if d.is_dir():
                    try:
                        n = sum(1 for _ in d.rglob("*.JPG"))
                    except Exception:
                        n = 0
                    out.append({"path": str(d), "label": d.name, "files": n})
    return out

@app.post("/api/index.cgi")
@app.get("/api/index.cgi")
@app.post("/api")
async def api(r: Request):
    try:
        b = await r.json()
    except Exception:
        b = {}
    b = b or {}
    act = b.get("action") or r.query_params.get("action") or "info"
    out = (b.get("output") or "").strip()

    if act == "info":
        return {"ok": True, "app": APP_NAME, "version": APP_VER, "mode": "local-agent",
                "script": str(SCRIPT), "python": py()}
    if act == "doctor":
        return {"ok": True, "result": json.loads(call("--doctor"))}
    if act == "devices":
        return {"ok": True, "list": list_devices()}
    if act == "mkdir":
        p = b.get("path", "")
        if not p:
            return {"ok": False, "error": "path 必填"}
        Path(p).mkdir(parents=True, exist_ok=True)
        return {"ok": True, "path": p}
    if act == "preview":
        if not b.get("source"):
            return {"ok": False, "error": "source 必填"}
        return {"ok": True, "result": json.loads(call(b["source"], "--output", out or ".", "--preview", *optargs(b)))}
    if act == "start":
```

```
        if not (b.get("source") and out):
            return {"ok": False, "error": "source 与 output 必填"}
        tid = b.get("taskId") or f"{time.strftime('%Y%m%d_%H%M%S')}_{random.randint(1000,9999)}"
        args = [b["source"], "--output", out, "--mode", b.get("mode", "copy"), "--task-id", tid, *optargs(b)]
        logdir = Path(out) / ".rawarchiver" / "tasks" / tid
        logdir.mkdir(parents=True, exist_ok=True)
        lf = open(logdir / "run.log", "a", encoding="utf-8")
        subprocess.Popen([py(), "-X", "utf8", str(SCRIPT), *[str(a) for a in args]],
                         stdout=lf, stderr=subprocess.STDOUT, cwd=str(ROOT))
        return {"ok": True, "taskId": tid}
    if act == "status":
        p = Path(out) / ".rawarchiver" / "tasks" / b.get("id", "") / "state.json"
        if not p.exists():
            return {"ok": False, "error": "任务不存在"}
        st = json.loads(p.read_text(encoding="utf-8"))
        st["logs"] = st.get("logs", [])[-80:]
        st.pop("done", None); st.pop("to_delete", None)
        return {"ok": True, "state": st}
    if act == "cancel":
        d = Path(out) / ".rawarchiver" / "tasks" / b.get("taskId", "")
        if not d.is_dir():
            return {"ok": False, "error": "任务不存在"}
        (d / "cancel").write_text("1", encoding="utf-8")
        return {"ok": True}
    if act == "cleanup":
        return {"ok": True, **json.loads(call("--output", out, "--task-id", b["taskId"], "--cleanup"))}
    if act == "rollback":
        return {"ok": True, **json.loads(call("--output", out, "--task-id", b["taskId"], "--rollback"))}
    return {"ok": False, "error": f"未知动作: {act}"}

def optargs(b):
    a = []
    if b.get("brandFolder"): a.append("--brand")
    if not b.get("handleJpg", True): a.append("--no-jpg")
    if not b.get("handleVideo", True): a.append("--no-video")
    if b.get("monthDir"): a.append("--month")
    a.append("--staging" if b.get("staging", True) else "--no-staging")
    return a

# 前端把 API 拼成 <路径>/api/index.cgi，这里同时挂静态站
app.mount("/", StaticFiles(directory=str(WWW), html=True), name="www")

if __name__ == "__main__":
    print(f"{APP_NAME} v{APP_VER} 本机 Agent: http://127.0.0.1:8765/")
    threading.Timer(1.2, lambda: webbrowser.open("http://127.0.0.1:8765/")).start()
    uvicorn.run(app, host="127.0.0.1", port=8765, log_level="warning")
```

scripts/pipeline.sh（一键流水线）

```
#!/bin/sh
# RAW 归档 —— 一键流水线（可挂 DSM 任务计划程序，以 root 执行以获得删卡权限）
# 用法：SOURCE=/volumeUSB1/usbshare OUTPUT=/volume1/photos/Camera MODE=move sh scripts/pipeline.sh
set -eu
ROOT="$(cd "$(dirname "$0")/.." && pwd)"
SOURCE="${SOURCE:-/volumeUSB1/usbshare}"
OUTPUT="${OUTPUT:-/volume1/photos/Camera}"
MODE="${MODE:-move}"
PYBIN="${PYBIN:-python3}"
EXTRA="${EXTRA:---brand --month}"
LOG="$ROOT/data/rawarchiver/pipeline.log"
mkdir -p "$(dirname "$LOG")" "$OUTPUT/.rawarchiver/tasks"

log(){ printf "[%s] %s\n" "$(date '+%Y-%m-%d %H:%M:%S')" "$*" | tee -a "$LOG"; }
die(){ log "ERROR: $*"; exit 1; }

[ -d "$SOURCE" ] || die "源不存在或未挂载: $SOURCE"
log "=== RAW 归档流水线启动 ==="
log "[0/4] 环境自检"
"$PYBIN" "$ROOT/scripts/raw_archiver.py" --doctor | tee -a "$LOG"

TID="$(date +%Y%m%d_%H%M%S)"

log "[1/4] 预览源 $SOURCE"
"$PYBIN" "$ROOT/scripts/raw_archiver.py" "$SOURCE" --output "$OUTPUT" --preview | tee -a "$LOG"

log "[2/4] 开始归档 (task=$TID mode=$MODE)"
"$PYBIN" "$ROOT/scripts/raw_archiver.py" "$SOURCE" --output "$OUTPUT" --mode "$MODE" \
  --task-id "$TID" $EXTRA || die "归档失败（可用 --task-id $TID 续跑）"
```

```
log "[3/4] 报告: $OUTPUT/_report_$TID.json"
[ -f "$OUTPUT/_report_$TID.json" ] || die "报告未生成"

log "[4/4] 触发下游钩子"
if [ -x "$ROOT/scripts/hooks/post_done.sh" ]; then
  "$ROOT/scripts/hooks/post_done.sh" "$TID" "$OUTPUT" >>"$LOG" 2>&1 || log "钩子返回非 0（忽略）"
else
  log "无 post_done 钩子，跳过"
fi
log "=== RAW 归档流水线结束 ==="
```

scripts/fetch_exiftool.sh（拉取 ExifTool）

```
#!/bin/sh
# RAW 归档 —— ExifTool 获取脚本（打包前跑一次，产物进 vendor/exiftool/）
# 三种来源，按顺序尝试：本机已有 → 官方 tar.bz2 → 直接下 .pl
set -e
ROOT="$(cd "$(dirname "$0")/.." && pwd)"
DEST="$ROOT/vendor/exiftool"
VER="${EXIFTOOL_VER:-13.00}"
URL="https://exiftool.org/Image-ExifTool-${VER}.tar.bz2"
mkdir -p "$DEST"

if [ -f "$DEST/exiftool" ]; then
  echo "已存在: $DEST/exiftool（跳过）。要覆盖请先删除该文件。"
  exit 0
fi

# 1) 本机已装的 ExifTool：直接复制自包含
if command -v exiftool >/dev/null 2>&1; then
  SRC="$(command -v exiftool)"
  echo "发现本机 ExifTool: $SRC"
  # 若是 perl 脚本，连 exiftool_files/lib 一起带上
  if head -c 40 "$SRC" | grep -qi perl; then
    cp "$SRC" "$DEST/exiftool"
    LIBDIR="$(dirname "$SRC")/../exiftool_files"
    [ -d "$LIBDIR/lib" ] && cp -a "$LIBDIR/lib" "$DEST/lib"
    [ -d "$(dirname $(readlink -f $SRC))/../exiftool_files/lib" ] && cp -a "$(dirname $(readlink -f $SRC))/../exiftool_files/lib" "$DEST/lib" 2>
/dev/null || true
  else
    cp -a "$SRC" "$DEST/exiftool"
  fi
  chmod +x "$DEST/exiftool"
  "$DEST/exiftool" -ver 2>/dev/null || perl "$DEST/exiftool" -ver
  echo "OK: 已从本机复制 ExifTool"
  exit 0
fi

# 2) 官方源码包
TMP="$(mktemp -d)"
echo "下载 $URL"
if command -v curl >/dev/null 2>&1; then curl -fL "$URL" -o "$TMP/et.tar.bz2"
elif command -v wget >/dev/null 2>&1; then wget -O "$TMP/et.tar.bz2" "$URL"
else echo "无 curl/wget，无法下载"; rm -rf "$TMP"; exit 1; fi

tar -xjf "$TMP/et.tar.bz2" -C "$TMP"
DIR="$(find "$TMP" -maxdepth 1 -type d -name 'Image-ExifTool-*' | head -1)"
[ -n "$DIR" ] || { echo "解压失败"; rm -rf "$TMP"; exit 1; }
cp -a "$DIR/exiftool" "$DEST/exiftool"
cp -a "$DIR/lib"      "$DEST/lib"
chmod +x "$DEST/exiftool"
rm -rf "$TMP"
perl "$DEST/exiftool" -ver && echo "OK: ExifTool $VER 已就位 -> $DEST"
```

scripts/hooks/post_done.sh（下游钩子）

```
#!/bin/sh
# RAW 归档 —— 完成后钩子（自定义下游动作：通知 / 索引 / 同步）
TID="$1"; OUT="$2"
REPORT="$OUT/_report_$TID.json"
[ -f "$REPORT" ] || { echo "report not found: $REPORT"; exit 1; }

# 1) 通知（填了环境变量才发）
if [ -n "${TG_BOT:-}" ] && [ -n "${TG_CHAT:-}" ]; then
  curl -s "https://api.telegram.org/bot${TG_BOT}/sendMessage" \
    -d chat_id="$TG_CHAT" -d text="RAW 归档完成 $TID" >/dev/null 2>&1 || true
fi
```

```
# 2) 下游示例：把报告交给索引程序（自行替换）
# /usr/local/bin/my_indexer --report "$REPORT"

# 3) 下游示例：挑片目录软链
# ln -sfn "$OUT" /volume1/homes/you/photos_latest

echo "[post_done] $TID -> $REPORT"
```

scripts/setup.py（依赖安装）

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
RAW 归档 — 依赖安装
优先使用包内离线 wheels（vendor/wheels），无网络时不至于卡住。
用法：python3 scripts/setup.py [--system]
"""
import os, sys, subprocess, sysconfig
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
VENV = ROOT / "agent" / "venv"
WHEELS = ROOT / "vendor" / "wheels"
REQ = ["exifread>=3.0", "Pillow>=10.0"]

def run(cmd):
    print("+ " + " ".join(cmd), flush=True)
    return subprocess.call(cmd)

def main():
    use_system = "--system" in sys.argv
    pip = [sys.executable, "-m", "pip"]
    if not use_system:
        if not VENV.exists():
            print("创建虚拟环境:", VENV)
            if run([sys.executable, "-m", "venv", str(VENV)]) != 0:
                print("venv 创建失败，回退 --system 安装")
                return main_system()
        pip = [str(VENV / ("Scripts/pip.exe" if os.name == "nt" else "bin/pip"))]
    args = pip + ["install", "--disable-pip-version-check"]
    if WHEELS.exists() and any(WHEELS.iterdir()):
        print("使用离线 wheels:", WHEELS)
        args += ["--no-index", "--find-links", str(WHEELS)]
    args += REQ
    rc = run(args)
    if rc != 0 and WHEELS.exists():
        print("离线安装失败，尝试在线安装 ...")
        rc = run(pip + ["install", "--disable-pip-version-check"] + REQ)
    print("依赖安装结果:", "OK" if rc == 0 else "FAILED")
    return rc

def main_system():
    rc = run([sys.executable, "-m", "pip", "install", "--disable-pip-version-check"] + REQ)
    return rc

if __name__ == "__main__":
    sys.exit(main())
```

build_all.sh（统一出包）

```
#!/bin/sh
# RAW 归档 v1.0.0 — 统一打包（SPK for DSM / FPK for fnOS / 源码包）
set -eu
VER=1.0.0; NAME=RAWArchiver
cd "$(dirname "$0")"
ROOT="$(pwd)"
DIST="$ROOT/dist"; STAGE="$ROOT/.stage"
rm -rf "$DIST" "$STAGE"; mkdir -p "$DIST" "$STAGE/spk" "$STAGE/fpk"

say(){ printf '==> %s\n' "$*"; }
die(){ printf 'ERROR: %s\n' "$*" >&2; exit 1; }

say "校验必需文件"
for f in scripts/raw_archiver.py app/rawarchiver/www/index.html app/rawarchiver/api/index.cgi \
         agent/agent.py pkg/spk/INFO pkg/fpk/package.conf; do
  [ -f "$f" ] || die "缺少 $f"
done
chmod +x scripts/raw_archiver.py app/rawarchiver/api/index.cgi agent/agent.py scripts/*.sh 2>/dev/null || true
```

```
python3 -c "import ast;ast.parse(open('scripts/raw_archiver.py',encoding='utf-8').read())" || die "raw_archiver.py 语法错误"
perl -c app/rawarchiver/api/index.cgi >/dev/null 2>&1 || say "提示：本机无 perl，跳过 CGI 语法检查"

if [ ! -f vendor/exiftool/exiftool ]; then
  if [ "${ALLOW_NO_EXIFTOOL:-0}" = "1" ]; then
    say "警告：无 ExifTool（ALLOW_NO_EXIFTOOL=1），视频日期会退化为文件时间"
  else
    die "缺 vendor/exiftool/exiftool，先执行: sh scripts/fetch_exiftool.sh （或 ALLOW_NO_EXIFTOOL=1 强行打包）"
  fi
fi

fill(){
  OUT="$1"
  mkdir -p "$OUT/app"
  cp -a app/rawarchiver "$OUT/app/rawarchiver"
  cp -a scripts         "$OUT/scripts"
  cp -a agent           "$OUT/agent"
  if [ -d vendor/exiftool ]; then mkdir -p "$OUT/vendor"; cp -a vendor/exiftool "$OUT/vendor/exiftool"; fi
  if [ -d vendor/wheels  ]; then mkdir -p "$OUT/vendor"; cp -a vendor/wheels  "$OUT/vendor/wheels"; fi
  cp -f requirements.txt "$OUT/" 2>/dev/null || true
  find "$OUT" -name '.DS_Store' -delete 2>/dev/null || true
  find "$OUT" -name '__pycache__' -type d -exec rm -rf {} + 2>/dev/null || true
}
say "组装 SPK 载荷"; fill "$STAGE/spk"
say "组装 FPK 载荷"; fill "$STAGE/fpk"

cp -a pkg/spk/INFO "$STAGE/spk/INFO"
cp -a pkg/spk/conf "$STAGE/spk/conf"
mkdir -p "$STAGE/spk/package-scripts.d" "$STAGE/spk/etc" "$STAGE/spk/var"
cp -a pkg/spk/package-scripts.d/install.sh "$STAGE/spk/package-scripts.d/install.sh"
chmod 755 "$STAGE/spk/package-scripts.d/install.sh" "$STAGE/spk/app/rawarchiver/api/index.cgi"

cp -a pkg/fpk/package.conf "$STAGE/fpk/package.conf"
cp -a pkg/fpk/cmd_script   "$STAGE/fpk/cmd_script"
chmod -R 755 "$STAGE/fpk/cmd_script"

SPK_ITEMS=$(cd "$STAGE/spk" && ls -A | tr '\n' ' ')
FPK_ITEMS=$(cd "$STAGE/fpk" && ls -A | tr '\n' ' ')
say "打 SPK: $SPK_ITEMS"; ( cd "$STAGE/spk" && tar -czf "$DIST/${NAME}-${VER}.spk" $SPK_ITEMS )
say "打 FPK: $FPK_ITEMS"; ( cd "$STAGE/fpk" && tar -czf "$DIST/${NAME}-${VER}.fpk" $FPK_ITEMS )
say "打源码包"; ( cd "$ROOT" && tar -czf "$DIST/${NAME}-${VER}-src.tar.gz" \
    --exclude='./dist' --exclude='./.stage' --exclude='./agent/venv' --exclude='*/__pycache__' . )

say "产物（sha256 前 16 位）"
( cd "$DIST" && for f in *; do printf '%s  %s\n' "$(sha256sum "$f" | cut -c1-16)" "$f"; done )
( cd "$DIST" && sha256sum * | grep -v SHA256SUMS > SHA256SUMS.txt )
say "完成：$DIST"
```

pkg/spk/INFO（DSM 套件元数据）

```
package="rawarchiver"
version="1.0.0"
displayname="RAW 归档"
description="RAW 归档：从 SD/TF/U 盘读取拍摄日期，按 YYYYMMDD 归档 RAW/JPG/视频"
description_chs="RAW 归档：从 SD/TF/U 盘读取拍摄日期，按 YYYYMMDD 归档 RAW/JPG/视频"
maintainer="you"
arch="noarch"
os_min_ver="7.0-41890"
dsmuidir="www"
dsmappname="SYNOCommunityDSMRawArchiver"
spawn_cron="false"
startable="yes"
```

pkg/fpk/package.conf（fnOS 包元数据）

```
{
  "name": "rawarchiver",
  "version": "1.0.0",
  "display_name": "RAW 归档",
  "desc": "从 SD/TF/U 盘读取拍摄日期，按 YYYYMMDD 归档 RAW/JPG/视频",
  "arch": "x86_64",
  "os_min_version": "0.9",
  "maintainer": "you",
  "startable": true,
  "run-as": "package",
  "service_dep": [],
  "resource-require": {"disk": "100M"}
}
```

requirements.txt

```
exifread>=3.0
Pillow>=10.0
```

agent/requirements.txt

```
fastapi>=0.104
uvicorn>=0.24
exifread>=3.0
Pillow>=10.0
```